

AZUKI · SECURITY REVIEW

Azuki

anime claimer.

A security review of anime claimer.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Azuki	anime claimer	Guardian	December 4 – December 17, 2024	December 24, 2024

§ 00

Contents.

34 pages — overview, confidence ranking, scope, 23 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 18 findings	P. 09
	Remediation Review · 5 findings	P. 28
§ 05	Disclaimer	P. 33
§ 06	About Guardian	P. 34

§ 01

Overview.

Guardian reviewed anime claimer across 2 review rounds. The final report includes 2 Critical, 1 High, 4 Medium, 16 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	chiru-labs-org/anime-claimer
CLIENT	METHODOLOGY
Azuki	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
anime claimer	8ed13d706dfd
PREPARED BY	
Guardian	
AUDIT WINDOW	
December 4 – December 17, 2024	
FINAL REPORT	
December 24, 2024	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	2	0	0	0	2
■ High	1	0	0	0	1
■ Medium	4	0	2	1	1
■ Low	16	0	10	1	5
■ Informational	0	0	0	0	0
All severities	23	0	12	2	9

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed anime claimer in `chiru-labs-org/anime-claimer`. The scope includes 382 source lines across 2 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
contracts/Timelock.sol	10	13
contracts/AnimeClaimer.sol	372	676

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Unable To Unpause Contract	Logical Error	CRITICAL	RESOLVED	p. 9
C-02	Permanent Claim DoS	Logical Error	CRITICAL	RESOLVED	p. 10
H-01	Missing Receive Configuration for Arbitrum	Config	HIGH	RESOLVED	p. 11
M-01	Sanctions Bypass With Delegate	Logical Error	MEDIUM	ACKNOWLEDGED	p. 13
M-02	Withdraw Limit Discrepancy between Chains	Logical Error	MEDIUM	RESOLVED	p. 14
M-03	Missing Validation In readyForClaim	Validation	MEDIUM	ACKNOWLEDGED	p. 15
M-04	Flash Loans Abused To Steal Vest	Gaming	MEDIUM	PARTIALLY RESOLVED	p. 16
L-01	Direct Boolean Check	Logical Error	LOW	RESOLVED	p. 17
L-02	Chain Inconsistencies Cause Failed Messages	Logical Error	LOW	ACKNOWLEDGED	p. 18
L-03	Variables Not Set In Setup Script	Warning	LOW	RESOLVED	p. 19
L-04	Zero Check for Signer	Warning	LOW	RESOLVED	p. 20
L-05	Underflow in Duration Calculation	Underflow	LOW	ACKNOWLEDGED	p. 21
L-06	Lacking tokenId Validation	Validation	LOW	ACKNOWLEDGED	p. 22
L-07	Smart Contract Claimers Warning	Warning	LOW	RESOLVED	p. 23

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-08	Unnecessary Signature Validation	Logical Error	LOW	ACKNOWLEDGED	p. 24
L-09	Vesting Root Can Be Re-Initialized	Logical Error	LOW	ACKNOWLEDGED	p. 25
L-10	Storage Variables Can't Be Read	Getters	LOW	PARTIALLY RESOLVED	p. 26
L-11	Optional DVNs Suggestion	Suggestion	LOW	RESOLVED	p. 27

REMEDIATION REVIEW

L-01	Lacking lastWithdrawnDay Getter	—	LOW	ACKNOWLEDGED	p. 28
L-02	Incorrect Comment	—	LOW	ACKNOWLEDGED	p. 29
L-03	Incorrect Function Name	—	LOW	ACKNOWLEDGED	p. 30
L-04	readyForClaim May Confuse Claimers	Logical Error	LOW	ACKNOWLEDGED	p. 31
L-05	Withdraw Limit Discrepancy Can Still Occur	—	LOW	ACKNOWLEDGED	p. 32

CRITICAL **C-01**

RESOLVED

Unable To Unpause Contract

CATEGORY	Logical Error
LOCATION	AnimeClaimer.sol: 391
REVIEW	Main Review

— DESCRIPTION

When the AnimeClaimer is initially created, the storage variable `paused` is set to true within the constructor: `_getAnimeClaimerStorage().paused = true;`

For the AnimeClaimer to become unpaused, it is expected the owner properly configures the contract's other parameters such as `dailyTotalWithdrawnLimit` and then calls function `setPaused(false)`. The issue is that when attempting to unpause the contract, the code also checks `if (!readyForClaim())`.

Function `readyForClaim` checks whether the contract is paused, and if so, returns false: `if ($.paused) return false;` Since the initial state is paused, `if (!readyForClaim())` will be entered. Therefore when the owner or manager attempts to unpause the contract, it reverts with `revert NotReadyForClaim();` Ultimately the entire contract's functionality is DoS'd as it is not possible to unpause the contract.

— RECOMMENDATION

When unpausing, do not revert if the contract is currently paused. Contract configuration parameters can be verified to be set outside of `readyForClaim` to avoid `if ($.paused) return false;`

— RESOLUTION

Status: Resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/14>

CRITICAL C-02

RESOLVED

Permanent Claim DoS

CATEGORY	Logical Error
LOCATION	AnimeClaimer.sol
REVIEW	Main Review

DESCRIPTION

The `_claimBatch` function prevents withdrawals once the daily withdrawal limit has been met: `if ($dailyTotalWithdrawnLimit <= $dailyTotalWithdrawn) revert ExceedDailyTotalWithdrawnLimit();`

The issue is that the `dailyTotalWithdrawn` is only reset within function `_incrementDailyTotalWithdrawn`, but `_incrementDailyTotalWithdrawn` is only called when a user withdraws a non-zero amount: `if (totalWithdrawn != 0)`

This leads to a total DoS once a user withdraws to the daily limit, as no more withdrawals are possible and the daily limit cannot be reset, even if it is a new day.

RECOMMENDATION

Update the daily limit regardless of whether a withdraw has occurred. Consider checking if the block timestamp is a new day and reset the `lastWithdrawnDay` or simply call `_incrementDailyTotalWithdrawn(0)` at the beginning of `claimBatch`.

RESOLUTION

Status: Resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/16>

HIGH **H-01**

RESOLVED

Missing Receive Configuration for Arbitrum

CATEGORY	Config
LOCATION	layerzero.mainnets.config.ts
REVIEW	Main Review

— DESCRIPTION

In order for messaging to work, configurations should be handled properly. According to LayerZero documentation:

> For a configuration to be considered correct, the Send Library configurations on Chain A must match Chain B's Receive Library configurations for filtering messages.

However as we can see from the configuration file, while the Send Library for source chain is configured, Receive Library for destination chain is not configured, hence the default values will be used for it in Arbitrum.

While the `confirmations` configured for Ethereum is "8", the default `confirmations` in Arbitrum is configured as "15", and this will lead to "Block Confirmation Mismatch" explained as:

> Messages will be blocked until either the sending OApp has increased the outbound block confirmations, or the receiving OApp decreases the inbound block confirmation threshold.

in LayerZero documentation.

Also default DVN's for Arbitrum are 'Layerzero Labs' and 'Google Cloud', hence other DVN's configured for source chain ('Nethermind', 'Horizen') won't be utilized as expected and they will just consume fee in source chain.

— RECOMMENDATION

Implement a proper configuration for destination chain's Receiver Library to match the Sender's. While doing so following documentation from LayerZero can help:

<https://docs.layerzero.network/v2/developers/evm/create-lz-oapp/configuring-pathways>

Be aware of following mismatch scenarios described and be sure Configurations in both end Matches:

<https://docs.layerzero.network/v2/developers/evm/protocol-gas-settings/default-config#debugging-configurations>

H-01 Missing Receive Configuration for Arbitrum

CONTINUED

For example, the following `receiveConfig` blob can be used for the arbitrum → ethereum connection:

```
{
  from: arbitrumContract,
  to: ethereumContract,
  config: {
    receiveConfig: {
      ulnConfig: {
        // SendUlnConfig and ReceiveUlnConfig confirmations must match
        confirmations: BigInt(8),
        requiredDVNs: [
          // Layerzero, Arbitrum
          '0x2f55c492897526677c5b68fb199ea31e2c126416',
          // Google, Arbitrum
          '0xd56e4eab23cb81f43168f9f45211eb027b9ac7cc',
          // Nethermind, Arbitrum
          '0xa7b5189bca84cd304d8553977c7c614329750d99',
          // Horizen, Arbitrum
          '0x19670df5e16bea2ba9b9e68b48c054c5baea06b8',
        ],
        optionalDVNs: [],
        optionalDVNThreshold: 0,
      },
    },
  },
}
```

— RESOLUTION

Status: Resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/22>

MEDIUM **M-01**

ACKNOWLEDGED

Sanctions Bypass With Delegate

CATEGORY Logical Error
LOCATION AnimeClaimer.sol: 496
REVIEW Main Review

DESCRIPTION

`if (_SANCTIONS_LIST.isSanctioned(msg.sender) || _SANCTIONS_LIST.isSanctioned(to))` is intended to prevent sanctioned addresses from using the AnimeClaimer. However, this protection can be bypassed simply with a sanctioned NFT owner delegating to a non-sanctioned address. That non-sanctioned address can then claim without issues.

RECOMMENDATION

Consider adding a sanctions list validation if the `msg.sender` is a delegate of the NFT owner.

Keep in mind that users can always transfer the NFT to another address, enabling them to claim the allocation.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

Withdraw Limit Discrepancy between Chains

CATEGORY Logical Error
LOCATION AnimeClaimer.sol
REVIEW Main Review

DESCRIPTION

The variable `dailyTotalWithdrawnLimit` can vary between chains. And indeed according to the runbook, the L1 limit is $1e24$, while the L2 limit is currently $1.5e23$. However, this discrepancy poses a problem where a message sent from the source chain (L1) may not be received in the destination chain (L2). This issue arises when the allocation sent is larger than L2's `dailyTotalWithdrawnLimit` but smaller than L1's `dailyTotalWithdrawnLimit`.

Since it is smaller than the L1 limit, it can be sent from the source chain, but because it exceeds the L2 limit, the message becomes stuck at the endpoint and cannot be received by the `AnimeClaimer` contract in L2.

RECOMMENDATION

Make sure the `dailyTotalWithdrawnLimit` value for L2 is greater or equal to the value set in L1 contract.

RESOLUTION

Status: Resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/18>

MEDIUM **M-03**

ACKNOWLEDGED

Missing Validation In readyForClaim

CATEGORY Validation
LOCATION AnimeClaimer.sol#L264
REVIEW Main Review

DESCRIPTION

`AnimeClaimer.readyForClaim` is a helper function that ensures the contract is correctly set up to initiate user claims.

However, the function is missing crucial validations:

Ethereum

- OApp correctly wired (peers, endpoint, delegate)

Arbitrum

- OApp correctly wired (peers, endpoint, delegate)
- Existing `ANIME` token balance

RECOMMENDATION

Consider adding the validation above to ensure both contracts are ready to start the claiming process and avoid failed transactions and messages.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

PARTIALLY RESOLVED

Flash Loans Abused To Steal Vest

CATEGORY Gaming
LOCATION Global
REVIEW Main Review

— DESCRIPTION

There are several protocols which offer NFT flashloans following EIP 6682, which grant ownership of an NFT for a single transaction from a pool of NFTs.

These protocols will allow arbitrary users to flash loan their NFT and claim the corresponding ANIME tokens from the NFTs vest in the merkle root even though they are not the official owners of the NFT.

Additionally, notice that fixed term NFT loans allow the loaner to claim any outstanding vest for the NFT that has not yet been claimed by the owner.

— RECOMMENDATION

Be aware of this edge case for flash loan providers of NFTs which will receive an ANIME vest. If this is deemed to be an issue those providers can be warned ahead of time to withdraw their NFT from flash loan service before the claimer becomes active.

— RESOLUTION

Status: Partially resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/25>

LOW **L-01**

RESOLVED

Direct Boolean Check

CATEGORY Logical Error
LOCATION AnimeClaimer.sol: 392
REVIEW Main Review

DESCRIPTION

`if (paused == false)` can just be rewritten to `!paused` within function `setPaused` to be consistent with other boolean operations.

RECOMMENDATION

Replace it with `!paused`.

RESOLUTION

Status: Resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/15>

LOW **L-02**

ACKNOWLEDGED

Chain Inconsistencies Cause Failed Messages

CATEGORY Logical Error
LOCATION AnimeClaimer.sol
REVIEW Main Review

DESCRIPTION

If the contract was paused on Arbitrum but not on ETH, a user could still send a claim request which would increment the total withdrawn on ETH, yet the withdraw would revert on the destination chain with `revert Paused();` Consequently, the `to` address will not receive their tokens and the request will have to be retried at the Endpoint after an unpause. This may be unexpected to users who had a successful `claimBatch` call.

RECOMMENDATION

Clearly document this behavior to users.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

RESOLVED

Variables Not Set In Setup Script

CATEGORY	Warning
LOCATION	AnimeClaimerSetup.s.sol
REVIEW	Main Review

— DESCRIPTION

The `manager` variable is not set for Arbitrum and `uuidSigner` is not set for ETH within `AnimeClaimerSetup` but other key parameters are. However this is not a major issue since setters are available within the AnimeClaimer contract.

— RECOMMENDATION

Be aware of this and considering setting those variables within the script. Also consider checking if those parameters should be set within `readyForClaim`.

— RESOLUTION

Status: Resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/19>

LOW **L-04**

RESOLVED

Zero Check for Signer

CATEGORY Warning
LOCATION AnimeClaimer.sol:L428
REVIEW Main Review

DESCRIPTION

The `setUUIDSigner` function does not limit the signer address and allows zero as a valid address. If the owner changes the signer address to zero by mistake, it will result in every signature check passing in `_claimBatch`, and the consequences can be catastrophic even if the mistake is resolved promptly upon discovery.

RECOMMENDATION

If the provided signer address is zero, the `setUUIDSigner` function should revert.

RESOLUTION

Status: Resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/20>

LOW **L-05**

ACKNOWLEDGED

Underflow in Duration Calculation

CATEGORY Underflow
LOCATION AnimeClaimer.sol:L597
REVIEW Main Review

— DESCRIPTION

According to the runbook, if the intention is to claim instantly, one can set the start and end times to the past, resulting in the whole allocation amount being returned optimistically by the `vested` function. However, if the start time is set to be greater than the end time, an underflow will occur in the duration calculation before comparing the start and end times with `block.timestamp` to return the total allocation, leading to a direct revert and preventing that claim.

— RECOMMENDATION

Ensure the end time greater than the start time in the root creation process.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Lacking tokenId Validation

CATEGORY	Validation
LOCATION	AnimeClaimer.sol: 486
REVIEW	Main Review

— DESCRIPTION

When claiming for a collector level claim the tokenId should not be assigned to a nonzero value.

This will be validated in the merkle proof, however out of an abundance of caution it may be prudent to add explicit validations at the contract level that the `tokenId` is 0 when `isForCollector` is true.

— RECOMMENDATION

Consider adding validation that enforces the `tokenId` to be 0 when `isForCollector` is true.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

RESOLVED

Smart Contract Claimers Warning

CATEGORY Warning
LOCATION AnimeClaimer.sol: 643
REVIEW Main Review

DESCRIPTION

The `_requestReleasePayload` function assigns the `msg.sender` as the refund receiver for a native refund. As a result, Smart Contracts which receive a collector allocation, do not have the functionality to delegate, and do not have a receive function cannot claim their vest.

RECOMMENDATION

This scenario is unlikely to occur, especially for a contract which is able to claim from the claimer. This finding simply serves as a warning for this case.

RESOLUTION

Status: Resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/23>

LOW **L-08**

ACKNOWLEDGED

Unnecessary Signature Validation

CATEGORY Logical Error
LOCATION AnimeClaimer.sol#L541
REVIEW Main Review

DESCRIPTION

When claiming allocation for Elementals, the `uuid` is defined and a signature verification is needed using the `uuidSigner`.

However, the first time the user claims allocation for an Elemental, besides checking the signature, the `uuid` is stored in a mapping:

```
$.nftToUUID[c.nft][c.tokenId] = c.uuid;
```

Therefore, the consecutive claims do not need additional signature verification, as the mapping will ensure its a valid claim:

```
if (c.uuid != $.nftToUUID[c.nft][c.tokenId]) revert InvalidUuid();
```

RECOMMENDATION

Consider validating the signature only when the mapping is updated with the new `uuid`.

RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Vesting Root Can Be Re-Initialized

CATEGORY Logical Error
LOCATION AnimeClaimer.sol#L399
REVIEW Main Review

— DESCRIPTION

The owner of `AnimeClaimer` can set the vesting root for the merkle distribution.

However, the function does not prevent the reinitialization of the vesting root. Due to the fact that the snapshot is only taken once, there is no need to leave the option for modifying the value again, as it will lead to accounting issues.

— RECOMMENDATION

Only allow the vesting root to be set once, or include it as a param in the constructor.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

PARTIALLY RESOLVED

Storage Variables Can't Be Read

CATEGORY	Getters
LOCATION	AnimeClaimer.sol#L99
REVIEW	Main Review

DESCRIPTION

The `AnimeClaimer` contract defines a struct for all storage variables. Although there are some getter functions to read this variables, there are some still missing that will provide a better user experience:

- `lastWithdrawnDay`
- `dailyTotalWithdrawn`

RECOMMENDATION

Adding these getters will allow the frontend to correctly display the current contract stats and limits.

RESOLUTION

Status: Partially resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/17>

LOW **L-11**

RESOLVED

Optional DVNs Suggestion

CATEGORY	Suggestion
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The current DVN configuration uses 4 required DVNs, however if there is an outage in any of the 4 DVNs this can cause a backlog of claim messages to build up and eventually be executed all in a single day, exceeding the daily limit.

This can result in DoS'd claims for other users for a period of time.

— RECOMMENDATION

Consider moving to a setup of optional DVNs whereby for example 4/5 or 3/4 DVNs are required to verify a message before execution. This way no single DVN can prevent a claim message from being executed.

— RESOLUTION

Status: Resolved.

<https://github.com/chiru-labs-org/anime-claimer/pull/22>

LOW **L-01**

ACKNOWLEDGED

Lacking lastWithdrawnDay Getter

CATEGORY —

LOCATION AnimeClaimer.sol

REVIEW Remediation Review

DESCRIPTION

The `AnimeClaimer` contract does not implement a getter function for the `lastWithdrawnDay` value.

RECOMMENDATION

Consider whether this variable should be exposed through it's own getter function similarly to other storage variables.

RESOLUTION

Status: Acknowledged.

<https://github.com/chiru-labs-org/anime-claimer/pull/31>

LOW **L-02**

ACKNOWLEDGED

Incorrect Comment

CATEGORY —

LOCATION layerzero.mainnets.config.ts: 83

REVIEW Remediation Review

DESCRIPTION

In the comment for the LayerZero receive configurations on Arbitrum it is mentioned that the receive library configured is `ReceiveUln302 for Ethereum`.

However the address provided is correctly the `ReceiveUln302` library address on Arbitrum.

RECOMMENDATION

Correct the comment to `ReceiveUln302 for Arbitrum`.

RESOLUTION

Status: Acknowledged.

<https://github.com/chiru-labs-org/anime-claimer/pull/31>

LOW **L-03**

ACKNOWLEDGED

Incorrect Function Name

CATEGORY —
LOCATION AnimeClaimerSetup.s.sol#L68
REVIEW Remediation Review

DESCRIPTION

The `AnimeClaimerL1Setup` performs the setup required for `AnimeClaimer` in Ethereum.

There is an inconsistency in the function name for `setUUIDSigner` as the script uses `setUuidSigner`, causing a revert when its executed.

RECOMMENDATION

Consider changing `setUuidSigner` for the correct function name in the contract, `setUUIDSigner`.

RESOLUTION

Status: Acknowledged.

<https://github.com/chiru-labs-org/anime-claimer/pull/31>

LOW **L-04**

ACKNOWLEDGED

readyForClaim May Confuse Claimers

CATEGORY	Logical Error
LOCATION	AnimeClaimer.sol#L291
REVIEW	Remediation Review

— DESCRIPTION

The `readyForClaim` function signals when the contract has been properly set up for claiming. However, the `paused` state is not included in this function, or in the `fullyReadyForClaim` function.

Users may become confused when reading the returned values of these functions, as even if both return true, the contract is not actually ready for claiming until the owner unpauses it.

Furthermore, the UI will need to combine both the `readyForClaim` and `isPaused` getters to enable claiming.

— RECOMMENDATION

Consider adding the paused state to the `fullyReadyForClaim` function, and return false if the contract is paused.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Withdraw Limit Discrepancy Can Still Occur

CATEGORY —
LOCATION AnimeClaimer.sol
REVIEW Remediation Review

— DESCRIPTION

Although M-02 was fixed in deployment, the setters in the contract do not validate that the L2 withdrawal limit is larger than the L1 limit.

— RECOMMENDATION

Consider adding validation in `setDailyTotalWithdrawnLimit` on the `limit` parameter depending on the chain id.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.